5 Scheduling

First Things First

> *How we spend our days is, of course, how we spend our lives.*
>
> —ANNIE DILLARD

> *"Why don't we write a book on scheduling theory?" I asked…."It shouldn't take much time!" Book-writing, like war-making, often entails grave miscalculations. Fifteen years later,* Scheduling *is still unfinished.*
>
> —EUGENE LAWLER

It's Monday morning, and you have an as-yet blank schedule and a long list of tasks to complete. Some can be started only after others are finished (you can't load the dishwasher unless it's unloaded first), and some can be started only after a certain time (the neighbors will complain if you put the trash out on the curb before Tuesday night). Some have sharp deadlines, others can be done whenever, and many are fuzzily in between. Some are urgent, but not important. Some are important, but not urgent. "We are what we repeatedly do," you seem to recall Aristotle saying—whether it's mop the floor, spend more time with family, file taxes on time, learn French.

So what to do, and when, and in what order? Your life is waiting.

Though we always manage to find *some* way to order the things we do in our days, as a rule we don't consider ourselves particularly good at it—hence the perennial bestseller status of time-management guides. Unfortunately, the guidance we find in them is frequently divergent and inconsistent. *Getting Things Done* advocates a policy of immediately doing any task of two minutes or less as soon as it comes to mind. Rival bestseller *Eat That Frog!* advises beginning with the most difficult task

and moving toward easier and easier things. *The Now Habit* suggests first scheduling one's social engagements and leisure time and then filling the gaps with work—rather than the other way around, as we so often do. William James, the "father of American psychology," asserts that "there's nothing so fatiguing as the eternal hanging on of an uncompleted task," but Frank Partnoy, in *Wait,* makes the case for deliberately *not* doing things right away.

Every guru has a different system, and it's hard to know who to listen to.

Spending Time Becomes a Science

Though time management seems a problem as old as time itself, the science of scheduling began in the machine shops of the industrial revolution. In 1874, Frederick Taylor, the son of a wealthy lawyer, turned down his acceptance at Harvard to become an apprentice machinist at Enterprise Hydraulic Works in Philadelphia. Four years later, he completed his apprenticeship and began working at the Midvale Steel Works, where he rose through the ranks from lathe operator to machine shop foreman and ultimately to chief engineer. In the process, he came to believe that the time of the machines (and people) he oversaw was not being used very well, leading him to develop a discipline he called "Scientific Management."

Taylor created a planning office, at the heart of which was a bulletin board displaying the shop's schedule for all to see. The board depicted every machine in the shop, showing the task currently being carried out by that machine and all the tasks waiting for it. This practice would be built upon by Taylor's colleague Henry Gantt, who in the 1910s developed the Gantt charts that would help organize many of the twentieth century's most ambitious construction projects, from the Hoover Dam to the Interstate Highway System. A century later, Gantt charts still adorn the walls and screens of project managers at firms like Amazon, IKEA, and SpaceX.

Taylor and Gantt made scheduling an object of study, and they gave it visual and conceptual form. But they didn't solve the fundamental problem of determining which schedules were best. The first hint that

this problem even *could* be solved wouldn't appear until several decades later, in a 1954 paper published by RAND Corporation mathematician Selmer Johnson.

The scenario Johnson examined was bookbinding, where each book needs to be printed on one machine and then bound on another. But the most common instance of this two-machine setup is much closer to home: the laundry. When you wash your clothes, they have to pass through the washer and the dryer in sequence, and different loads will take different amounts of time in each. A heavily soiled load might take longer to wash but the usual time to dry; a large load might take longer to dry but the usual time to wash. So, Johnson asked, if you have several loads of laundry to do on the same day, what's the best way to do them?

His answer was that you should begin by finding the single step that takes the least amount of time—the load that will wash *or* dry the quickest. If that shortest step involves the washer, plan to do that load *first*. If it involves the dryer, plan to do it *last*. Repeat this process for the remaining loads, working from the two ends of the schedule toward the middle.

Intuitively, Johnson's algorithm works because regardless of how you sequence the loads, there's going to be some time at the start when the washer is running but not the dryer, and some time at the end when the dryer is running but not the washer. By having the shortest washing times at the start, and the shortest drying times at the end, you maximize the amount of overlap—when the washer and dryer are running simultaneously. Thus you can keep the total amount of time spent doing laundry to the absolute minimum. Johnson's analysis had yielded scheduling's first optimal algorithm: start with the lightest wash, end with the smallest hamper.

Beyond its immediate applications, Johnson's paper revealed two deeper points: first, that scheduling could be expressed algorithmically, and second, that optimal scheduling solutions existed. This kicked off what has become a sprawling literature, exploring strategies for a vast menagerie of hypothetical factories with every conceivable number and kind of machines.

We're going to focus on a tiny subset of this literature: the part that, unlike bookbinding or laundry, deals with scheduling for a *single*

machine. Because the scheduling problem that matters the most involves just one machine: ourselves.

Handling Deadlines

With single-machine scheduling, we run into something of a problem right off the bat. Johnson's work on bookbinding was based on minimizing the total time required for the two machines to complete all of their jobs. In the case of single-machine scheduling, however, if we are going to do all the tasks assigned, then all schedules will take equally long to complete; the order is irrelevant.

This is a sufficiently fundamental and counterintuitive point that it's worth repeating. If you have only a single machine, and you're going to do all of your tasks, then any ordering of the tasks will take you the same amount of time.

Thus we encounter the first lesson in single-machine scheduling literally before we even begin: *make your goals explicit*. We can't declare some schedule a winner until we know how we're keeping score. This is something of a theme in computer science: before you can have a plan, you must first choose a metric. And as it turns out, which metric we pick here will directly affect which scheduling approaches fare best.

The first papers on single-machine scheduling followed quickly on the heels of Johnson's bookbinding work and offered several plausible metrics to consider. For each metric, they discovered a simple, optimal strategy.

It is of course common, for instance, for tasks to have a due date, with the lateness of a task being how far it has gone overdue. So we can think of the "maximum lateness" of a set of tasks as the lateness of whatever task has gone furthest past its due date—the kind of thing your employer might care about in a performance review. (Or what your customers might care about in a retail or service setting, where the "maximally late" task corresponds to the customer subjected to the longest wait time.)

If you're concerned with minimizing maximum lateness, then the best strategy is to start with the task due soonest and work your way toward the task due last. This strategy, known as **Earliest Due Date**, is fairly intuitive. (For instance, in a service-sector context, where each arriving

patron's "due date" is effectively the instant they walk in the door, it just means serving customers in order of arrival.) But some of its implications are surprising. For example, how long each task will take to complete is entirely irrelevant: it doesn't change the plan, so in fact you don't even need to know. All that matters is when the tasks are due.

You might already be using Earliest Due Date to tackle your workload, in which case you probably don't need computer science to tell you that it's a sensible strategy. What you may not have known, though, is that it's the *optimal* strategy. More precisely, it is optimal assuming that you're only interested in one metric in particular: reducing your maximum lateness. If that's not your goal, however, then another strategy might be more applicable.

For instance, consider the refrigerator. If you're one of the many people who have a community-supported agriculture (CSA) subscription, then every week or two you've got a lot of fresh produce coming to your doorstep all at once. Each piece of produce is set to spoil on a different date—so eating them by Earliest Due Date, in order of their spoilage schedule, seems like a reasonable starting point. It's not, however, the end of the story. Earliest Due Date is optimal for reducing maximum lateness, which means it will minimize the rottenness of the *single most rotten thing* you'll have to eat; that may not be the most appetizing metric to eat by.

Maybe instead we want to minimize the *number* of foods that spoil. Here a strategy called **Moore's Algorithm** gives us our best plan. Moore's Algorithm says that we start out just like with Earliest Due Date —by scheduling out our produce in order of spoilage date, earliest first, one item at a time. However, as soon as it looks like we won't get to eating the next item in time, we pause, look back over the meals we've already planned, and *throw out* the biggest item (that is, the one that would take the most days to consume). For instance, that might mean forgoing the watermelon that would take a half dozen servings to eat; not even attempting it will mean getting to everything that follows a lot sooner. We then repeat this pattern, laying out the foods by spoilage date and tossing the largest already scheduled item any time we fall behind. Once everything that remains can be eaten in order of spoilage date without anything spoiling, we've got our plan.

Moore's Algorithm minimizes the number of items you'll need to throw away. Of course, you're also welcome to compost the food, donate it to the local food bank, or give it to your neighbor. In an industrial or bureaucratic context where you can't simply discard a project, but in which the *number*—rather than the severity—of late projects is still your biggest concern, Moore's Algorithm is just as indifferent about how those late tasks are handled. Anything booted from the main portion of your schedule can get done at the very end, in any order; it doesn't matter, as they're all already late.

Getting Things Done

> *Do the difficult things while they are easy and do the great things while they are small.*
>
> —LAO TZU

Sometimes due dates aren't our primary concern and we just want to get stuff done: as much stuff, as quickly as possible. It turns out that translating this seemingly simple desire into an explicit scheduling metric is harder than it sounds.

One approach is to take an outsider's perspective. We've noted that in single-machine scheduling, nothing we do can change how long it will take us to finish all of our tasks—but if each task, for instance, represents a waiting client, then there is a way to take up as little of *their* collective time as possible. Imagine starting on Monday morning with a four-day project and a one-day project on your agenda. If you deliver the bigger project on Thursday afternoon (4 days elapsed) and then the small one on Friday afternoon (5 days elapsed), the clients will have waited a total of 4 + 5 = 9 days. If you reverse the order, however, you can finish the small project on Monday and the big one on Friday, with the clients waiting a total of only 1 + 5 = 6 days. It's a full workweek for you either way, but now you've saved your clients three days of their combined time. Scheduling theorists call this metric the "sum of completion times."

Minimizing the sum of completion times leads to a very simple optimal algorithm called **Shortest Processing Time**: always do the quickest task you can.

Even if you don't have impatient clients hanging on every job, Shortest Processing Time gets things *done*. (Perhaps it's no surprise that it is compatible with the recommendation in *Getting Things Done* to immediately perform any task that takes less than two minutes.) Again, there's no way to change the total amount of time your work will take you, but Shortest Processing Time may ease your mind by shrinking the number of outstanding tasks as quickly as possible. Its sum-of-completion-times metric can be expressed another way: it's like focusing above all on reducing the length of your to-do list. If each piece of unfinished business is like a thorn in your side, then racing through the easiest items may bring some measure of relief.

Of course, not all unfinished business is created equal. Putting out an actual fire in the kitchen should probably be done before "putting out a fire" with a quick email to a client, even if the former takes a bit longer. In scheduling, this difference of importance is captured in a variable known as *weight*. When you're going through your to-do list, this weight might feel literal—the burden you get off your shoulders by finishing each task. A task's completion time shows how long you carry that burden, so minimizing the sum of *weighted* completion times (that is, each task's duration multiplied by its weight) means minimizing your total oppression as you work through your entire agenda.

The optimal strategy for that goal is a simple modification of Shortest Processing Time: divide the weight of each task by how long it will take to finish, and then work in order from the highest resulting importance-per-unit-time (call it "density" if you like, to continue the weight metaphor) to the lowest. And while it might be hard to assign a degree of importance to each one of your daily tasks, this strategy nonetheless offers a nice rule of thumb: only prioritize a task that takes twice as long if it's twice as important.

In business contexts, "weight" might easily be translated to the amount of money each task will bring in. The notion of dividing reward by duration translates, therefore, to assigning each task an hourly rate. (If you're a consultant or freelancer, that might in effect already be done for you: simply divide each project's fee by its size, and work your way from the highest hourly rate to the lowest.) Interestingly, this weighted strategy also shows up in studies of animal foraging, with nuts and berries taking the place of dollars and cents. Animals, seeking to

maximize the rate at which they accumulate energy from food, should pursue foods in order of the ratio of their caloric energy to the time required to get and eat them—and indeed appear to do so.

When applied to debts rather than incomes, the same principle yields a strategy for getting in the black that's come to be called the "debt avalanche." This debt-reduction strategy says to ignore the number and size of your debts entirely, and simply funnel your money toward the debt with the single highest interest rate. This corresponds rather neatly to working through jobs in order of importance per unit time. And it's the strategy that will reduce the total burden of your debt as quickly as possible.

If, on the other hand, you're more concerned with reducing the *number* of debts than the *amount* of debt—if, for instance, the hassle of numerous bills and collection phone calls is a bigger deal than the difference in interest rates—then you're back to the unweighted, "just get stuff done" flavor of Shortest Processing Time, paying off the smallest debts first simply to get them out of the way. In debt-reduction circles, this approach is known as the "debt snowball." Whether people, in practice, ought to prioritize lowering the dollar amount of their debts or the quantity of them remains an active controversy, both in the popular press as well as in economics research.

Picking Our Problems

This brings us back to the note on which we began our discussion of single-machine scheduling. It's said that "a man with one watch knows what time it is; a man with two watches is never sure." Computer science can offer us optimal algorithms for various metrics available in single-machine scheduling, but choosing the metric we want to follow is up to us. In many cases, we get to decide what problem we want to be solving.

This offers a radical way to rethink procrastination, the classic pathology of time management. We typically think of it as a faulty algorithm. What if it's exactly the opposite? What if it's an optimal solution *to the wrong problem*?

There's an episode of *The X-Files* where the protagonist Mulder, bedridden and about to be consumed by an obsessive-compulsive